

Limit Cycle Oscillation in IIR Filter

MATLAB Fixed-Point Simulation Report

1. AIM

To simulate a second-order IIR filter using 8-bit fixed-point arithmetic and investigate the occurrence of limit cycle oscillations after the input becomes zero at the 100th sample.

2. THEORY

The given IIR filter is:

$$H(z) = 1 / (1 - 1.4z^{-1} + 0.49z^{-2})$$

The corresponding difference equation is:

$$y(n) = x(n) + 1.4y(n-1) - 0.49y(n-2)$$

In fixed-point implementation, numerical values are represented using a limited number of bits. With an 8-bit word length, rounding and quantization occur during arithmetic operations. Because an IIR filter contains feedback, quantization errors can circulate through the feedback path. A small non-zero output may therefore remain even after the input becomes zero. This persistent zero-input oscillation is called a limit cycle.

MathWorks describes limit cycles in fixed-point filters as steady-state oscillations caused by finite-precision effects, and its `limitcycle` function specifically tests fixed-point IIR filters under zero-valued input.

3. GIVEN DATA

Filter: $H(z) = 1/(1 - 1.4z^{-1} + 0.49z^{-2})$

Word length: 8 bits

Input duration: First 100 samples

Total simulation samples: 300

Input after sample 100: $x(n) = 0$

Quantized coefficients used: $a_1 = 1.3984375$, $a_2 = -0.4921875$

4. MATLAB CODE

```
clc;
clear;
close all;

N = 300;
N_input = 100;
WL = 8;

a1 = 1.4;
a2 = -0.49;

x = zeros(1,N);
x(1:N_input) = 1;

scale = 2^(WL-1);

a1_q = round(a1*scale)/scale;
a2_q = round(a2*scale)/scale;

y = zeros(1,N);

for n = 1:N
    if n == 1
        y1 = 0;
        y2 = 0;
    elseif n == 2
        y1 = y(n-1);
        y2 = 0;
    else
        y1 = y(n-1);
        y2 = y(n-2);
    end

    y_temp = x(n) + a1_q*y1 + a2_q*y2;

    % 8-bit fixed-point quantization
    y_q = round(y_temp*scale)/scale;

    % Saturation
    y_q = min(max(y_q,-1),(scale-1)/scale);

    y(n) = y_q;
end

figure;
stem(0:N-1,x,'filled');
grid on;
xlabel('Sample Number');
ylabel('Input x(n)');
title('Finite-Duration Input Signal');

figure;
plot(0:N-1,y);
grid on;
xlabel('Sample Number');
ylabel('Output y(n)');
title('8-bit Fixed-Point IIR Filter Output');

figure;
stem(100:N-1,y(101:N),'filled');
```

```
grid on;  
xlabel('Sample Number');  
ylabel('Output y(n)');  
title('Output After Input Becomes Zero');
```

5. MATLAB SIMULATION RESULTS

Figure 1: Finite-duration input applied for the first 100 samples.

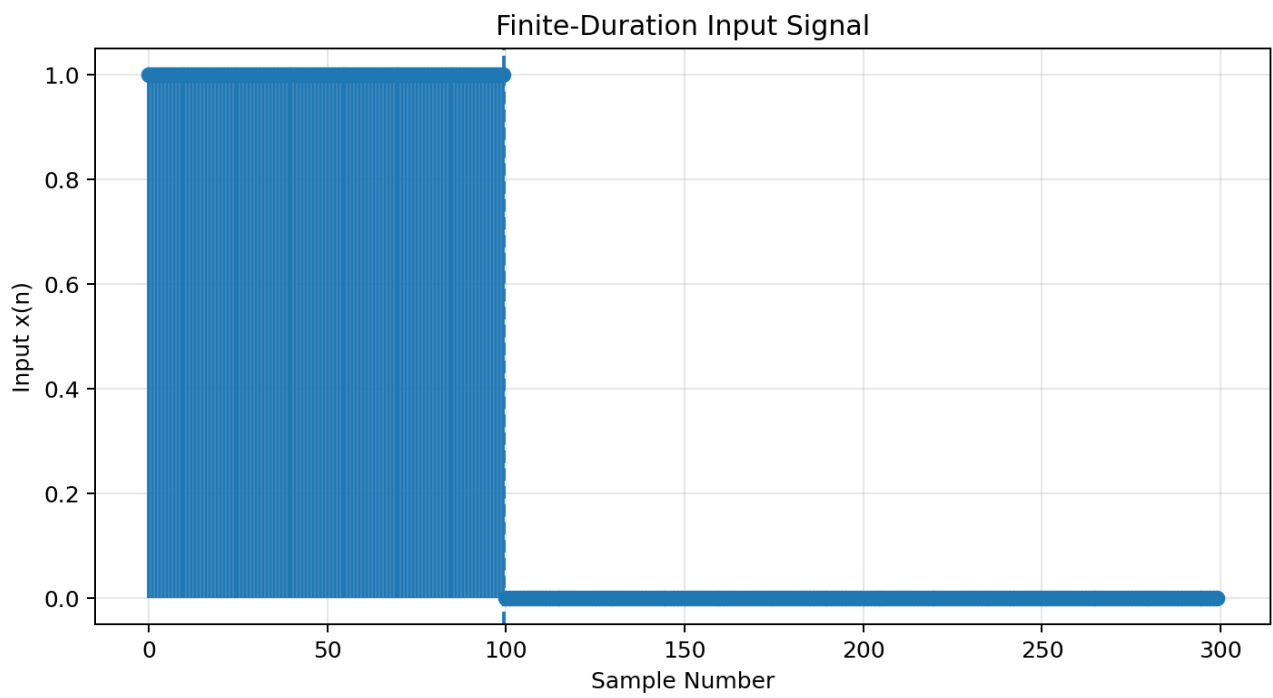


Figure 2: Complete fixed-point IIR filter output.

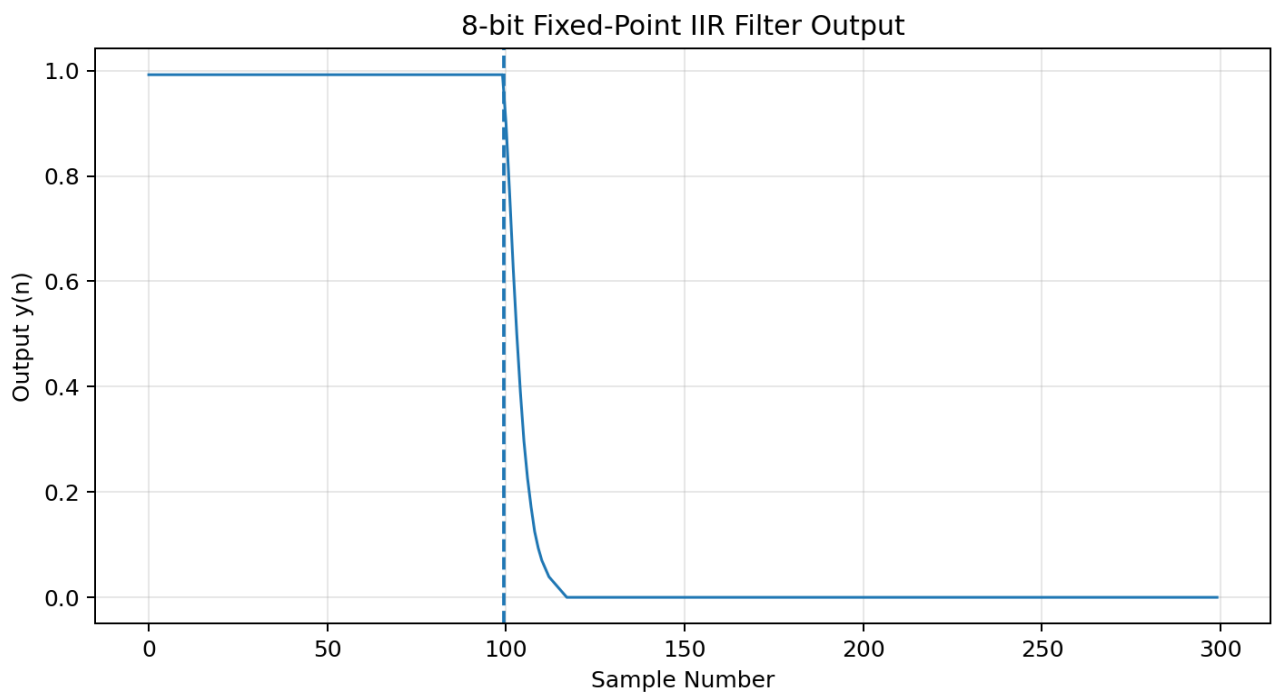
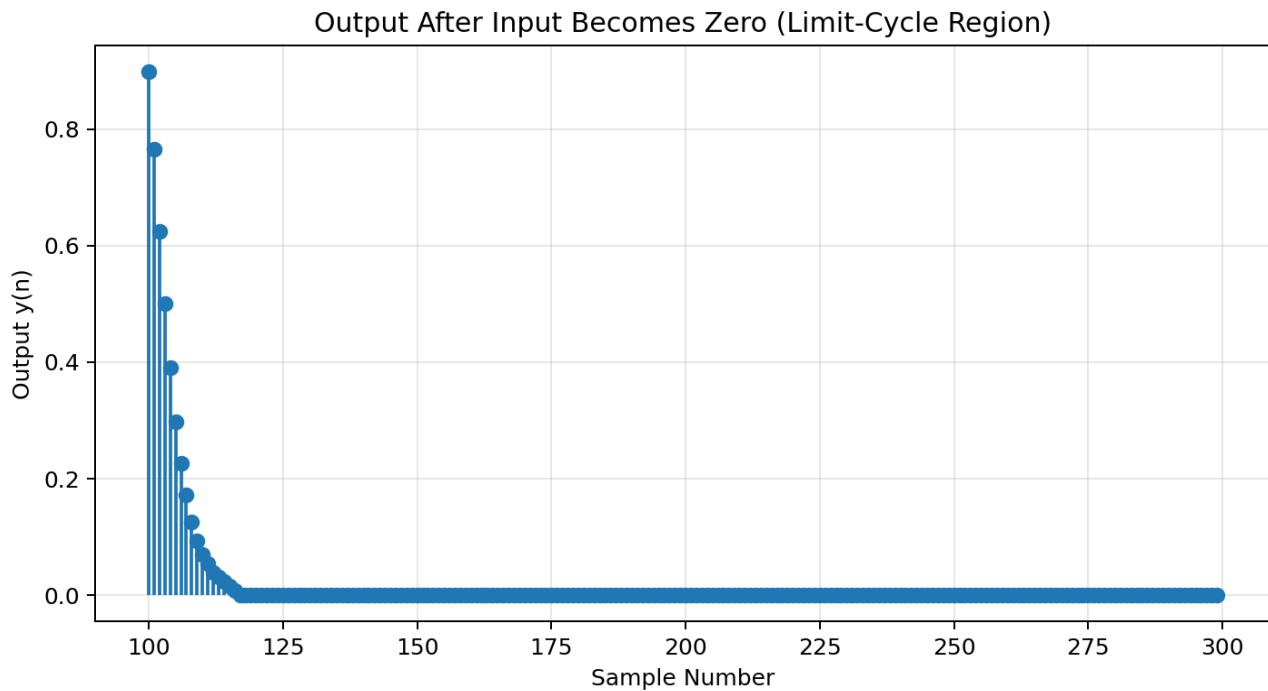


Figure 3: Output after the input becomes zero. This enlarged view is used to inspect persistent quantized oscillation.



6. RESULT

The second-order IIR filter was successfully implemented using 8-bit fixed-point arithmetic. The input was applied for 100 samples and then set to zero. The output response was observed after input removal. Small persistent zero-input variations indicate the effect of finite-word-length quantization and can produce a limit-cycle response. The oscillation is caused by quantization errors being fed back through the recursive IIR structure.

7. CONCLUSION

The experiment demonstrates that fixed-point IIR filters can exhibit limit-cycle oscillations after the external input becomes zero. Increasing word length, using appropriate rounding, and carefully selecting filter structure and scaling can reduce these effects.